

Unit 3

Class:

Defining classes

A class is a user-defined blueprint or template in Java that is used to create objects. It represents a group of properties (fields) and behaviors (methods) that are common to all objects of that type.

Syntax:

```
class ClassName {  
    // fields  
    // methods  
}
```

A class acts as a logical structure that binds data and methods together. It does not occupy memory until an object of the class is created. Classes support data abstraction and reusability in Java.

Example :

```
class Student {  
    int rollNo;  
    String name;  
  
    void display() {  
        System.out.println(rollNo + " " + name);  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.rollNo = 1;  
        s1.name = "Amit";  
        s1.display();  
    }  
}
```

Output:

1 Amit

Here in code:

1. A class Student is defined with fields and a method.
2. An object s1 is created using the new keyword.
3. Data members are accessed using the object reference.

Fields and methods

Definition:

Fields are variables declared inside a class, while methods are functions that define the behavior of the class.

Syntax:

```
class ClassName {  
    dataType fieldName;  
    returnType methodName() {  
        // method body  
    }  
}
```

Fields store object data and methods operate on that data. Fields can be instance or static, while methods help achieve modularity and code reuse.

Example :

```
class Employee {  
    int id;  
    String name;  
  
    void show() {  
        System.out.println(id + " " + name);  
    }  
  
    public static void main(String[] args) {  
        Employee e = new Employee();  
        e.id = 101;  
        e.name = "Ravi";  
        e.show();  
    }  
}
```

Output:

101 Ravi

1. Fields id and name store employee details.
2. Method show() prints field values.
3. Object e accesses fields and method.

Creating objects

Definition:

An object is an instance of a class created using the new keyword.

Syntax:

ClassName objectName = **new** ClassName();

Objects represent real-world entities and occupy memory. Multiple objects can be created from a single class.

Example :

```
class Car {  
    String model;  
  
    public static void main(String[] args) {  
        Car c1 = new Car();  
        c1.model = "Honda";  
        System.out.println(c1.model);  
    }  
}
```

Output:

Honda

In above Code :

1. Class Car is defined.
2. Object c1 is created.
3. Field model is accessed using object reference.

Accessing rules :

Accessing rules in Java define how the data members (fields) and methods of a class can be accessed using objects, class names, or within the same class. These rules ensure proper use of encapsulation and data security in object-oriented programming.

Syntax:

```
objectName.variableName;  
objectName.methodName();  
ClassName.staticVariable;  
ClassName.staticMethod();
```

Accessing rules depend on how members are declared and from where they are accessed. Instance variables and instance methods are accessed using object references, whereas static members are accessed using the class name. Private members are accessible only within the same class, while public members can be accessed from anywhere. These rules help control visibility and protect data from unauthorized access.

Example :

```

class Student {
    int rollNo;           // instance variable
    static String college = "ABC College"; // static variable

    void display() {      // instance method
        System.out.println(rollNo);
    }

    static void showCollege() { // static method
        System.out.println(college);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.rollNo = 10;      // accessing instance variable
        s.display();        // accessing instance method

        Student.showCollege(); // accessing static method
    }
}

```

Output:

```

10
ABC College

```

Here,

1. Instance members are accessed using the object s.
2. Static members are accessed using the class name Student.
3. The program demonstrates correct access of instance and static members.

Java Access Modifiers :

Java access modifiers are used to specify the scope of the variables, data members, methods, classes, or constructors. These help to restrict and secure the access (or, level of access) of the data.

There are four different types of access modifiers in Java, we have listed them as follows:

- Default (No keyword required)
- Private
- Protected
- Public

Default Access Modifier

Default access modifier means we do not explicitly declare an access modifier for a class, field, method, etc.

A variable or method declared without any access control modifier is available to any other class in the same package. The fields in an interface are implicitly public static final and the methods in an interface are by default public.

Example of Default Access Modifiers

Variables and methods can be declared without any modifiers, as in the following examples –

```
String version = "1.5.1";
```

```
boolean processOrder() {  
    return true;  
}
```

Private Access Modifier

Methods, variables, and constructors that are declared private can only be accessed within the declared class itself.

Private access modifier is the most restrictive access level. Class and interfaces cannot be private.

Variables that are declared private can be accessed outside the class, if public getter methods are present in the class.

Using the private modifier is the main way that an object encapsulates itself and hides data from the outside world.

Examples of Private Access Modifiers

Example 1

The following class uses private access control –

```
public class Logger {  
    private String format;  
    public String getFormat() {  
        return this.format;  
    }  
    public void setFormat(String format) {  
        this.format = format;  
    }  
}
```

```
}  
}
```

Here, the format variable of the Logger class is private, so there's no way for other classes to retrieve or set its value directly.

So, to make this variable available to the outside world, we defined two public methods: getFormat(), which returns the value of format, and setFormat(String), which sets its value.

Protected Access Modifier

Variables, methods, and constructors, which are declared protected in a superclass can be accessed only by the subclasses in other package or any class within the package of the protected members' class.

The protected access modifier cannot be applied to class and interfaces. Methods, fields can be declared protected, however methods and fields in a interface cannot be declared protected.

Protected access gives the subclass a chance to use the helper method or variable, while preventing a nonrelated class from trying to use it.

Examples of Protected Access Modifiers

Example 1

The following parent class uses protected access control, to allow its child class override openSpeaker() method –

```
class AudioPlayer {  
    protected boolean openSpeaker(Speaker sp) {  
        // implementation details  
    }  
}  
  
class StreamingAudioPlayer extends AudioPlayer {  
    boolean openSpeaker(Speaker sp) {  
        // implementation details  
    }  
}
```

Here, if we define openSpeaker() method as private, then it would not be accessible from any other class other than AudioPlayer. If we define it as public, then it would become

accessible to all the outside world. But our intention is to expose this method to its subclass only, that's why we have used protected modifier.

Public Access Modifier

A class, method, constructor, interface, etc. declared public can be accessed from any other class. Therefore, fields, methods, blocks declared inside a public class can be accessed from any class belonging to the Java Universe.

However, if the public class we are trying to access is in a different package, then the public class still needs to be imported. Because of class inheritance, all public methods and variables of a class are inherited by its subclasses.

Syntax

The following function uses public access control –

```
public static void main(String[] arguments) {  
    // ...  
}
```

The main() method of an application has to be public. Otherwise, it could not be called by a Java interpreter (such as java) to run the class.

Example of Public Access Modifiers

This example demonstrates the use of public access modifier.

```
// Class One
```

```
class One {  
    public void printOne() {  
        System.out.println("printOne method of One class.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Creating an object of class One  
        One obj = new One();  
        // Calling printOne() method of class One  
        obj.printOne();  
    }  
}
```

```
}
```

Output

This example demonstrates the use of public access modifier.

this keyword

The this keyword refers to the current object in a method or constructor.

The most common use of the this keyword is to eliminate the confusion between class attributes and parameters with the same name (because a class attribute is shadowed by a method or constructor parameter).

this can also be used to:

- Invoke current class constructor
- Invoke current class method
- Return the current class object
- Pass an argument in the method call
- Pass an argument in the constructor call

Syntax:

this.variableName;

It is mainly used to differentiate instance variables from local variables and to invoke current class constructors or methods.

Example Code:

```
class Person {  
    String name;  
  
    Person(String name) {  
        this.name = name;  
    }  
  
    void show() {  
        System.out.println(name);  
    }  
  
    public static void main(String[] args) {  
        Person p = new Person("Neha");  
        p.show();  
    }  
}
```


Output: Neha

In this Code:

1. Constructor parameter and instance variable have same name.
2. this.name refers to instance variable.
3. Object prints correct value.

Static keyword

The static keyword is a non-access modifier used for methods and attributes. Static methods/attributes can be accessed without creating an object of a class.

Syntax:

```
static dataType variableName;
```

Static members are shared among all objects and can be accessed without creating an object.

Example:

```
class College {  
    static String collegeName = "ABC College";  
  
    public static void main(String[] args) {  
        System.out.println(collegeName);  
    }  
}
```

Output:

ABC College

In above Code:

1. Static variable is declared.
2. No object is created.
3. Static member accessed directly.

Method overloading

Method overloading is a feature that allows a class to have multiple methods with the same name but different parameters. The compiler distinguishes between these methods based on their method signatures, which include the number, type, and order of parameters. This enables a single method name to perform different actions based on the arguments provided during the call.

Syntax:

```
methodName(int a)  
methodName(int a, int b)
```

It increases readability and supports compile-time polymorphism.

Example :

```
class MathOp {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        MathOp m = new MathOp();
        System.out.println(m.add(2, 3));
        System.out.println(m.add(1, 2, 3));
    }
}
```

Output:

5
6

Here :

1. Same method name used with different parameters.
2. Compiler decides method call at compile time.
3. Demonstrates method overloading.

Final keyword

The final keyword is used to restrict modifications.

final variable: A constant whose value cannot be changed after initialization.

final method: Cannot be overridden by subclasses.

final class: Cannot be inherited by other classes, preventing extension.

Used to **prevent changes**.

(a) final variable → constant

final int MAX = 100;

(b) final method → cannot be overridden

final void show() {}

(c) final class → cannot be inherited

final class Vehicle {}

Syntax:

final dataType variableName;

Final variables cannot be changed, final methods cannot be overridden, and final classes cannot be inherited.

Example :

```
class Test {  
    final int x = 10;  
  
    public static void main(String[] args) {  
        Test t = new Test();  
        System.out.println(t.x);  
    }  
}
```

Output:

10

In this Code:

1. x is declared as final.
2. Its value cannot be modified.
3. Ensures data security.

Constructors:

A constructor in Java is a special method used to initialize objects. It is automatically invoked when an object of a class is created. Constructors are mainly used to set initial values for object attributes and to perform any setup required at the time of object creation.

Default constructors

Default Constructors are constructors that do not accept any parameters. They are automatically invoked when an object is created without passing any arguments. If a programmer does not define any constructor in a class, the compiler automatically provides a default constructor.

The main purpose of a default constructor is to initialize instance variables with default values (such as 0, null, or false) and perform basic setup tasks. Default constructors help in simple object creation and ensure that an object is properly initialized even when no explicit values are provided.

Example:

```
class MyClass {  
    int data;  
    String name;  
}
```

```

public class Test {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        System.out.println(obj.data + " " + obj.name);
    }
}

```

Output:

0 null

Parameterized constructors

Parameterized Constructors are constructors that accept parameters at the time of object creation. These parameters are used to initialize the instance variables of a class with specific values, allowing objects to be created with predefined data.

Parameterized constructors provide greater control and flexibility over object initialization compared to default constructors. They ensure that an object is created in a valid and meaningful state by assigning required values during instantiation. This type of constructor is widely used in object-oriented programming languages such as Java, C++, and C#.

Example:

```

class MyClass {
    int data;
    String name;

    MyClass(int d, String n) {
        data = d;
        name = n;
    }
}

```

```

public class Test {
    public static void main(String[] args) {
        MyClass obj = new MyClass(10, "Amit");
        System.out.println(obj.data + " " + obj.name);
    }
}

```

Output:

10 Amit

Copy constructor

Copy Constructor is a special type of constructor used to create a new object by copying the data of an existing object of the same class. It receives an object as a parameter and initializes the new object's instance variables with the values of that object.

The main purpose of a copy constructor is to duplicate objects, ensuring that each object has its own separate memory while containing the same data. Copy constructors improve

data safety, clarity, and controlled copying of objects. This concept is commonly used in object-oriented programming languages such as C++, and in Java it is implemented using a constructor that takes an object of the same class as an argument.

Example:

```
class MyClass {
    int data;
    String name;

    MyClass(MyClass other) {
        this.data = other.data;
        this.name = other.name;
    }
}

public class Test {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(20, "Ravi");
        MyClass obj2 = new MyClass(obj1);
        System.out.println(obj2.data + " " + obj2.name);
    }
}
```

Output:

20 Ravi

Passing object as a parameter

Passing Object as a Parameter is an object-oriented programming concept in which an object of a class is passed as an argument to a method or constructor. Instead of passing individual values, the entire object is provided, allowing the method to access and use the object's data members and methods through reference.

This technique promotes code reusability, modularity, and efficient data handling, especially when multiple related values need to be processed together. In languages like Java, objects are passed by reference (value of reference), so changes made to the object inside the method can affect the original object. Passing objects as parameters is commonly used in operations such as comparison, copying data, and object interaction.

Example:

```
class AnotherClass {
    MyClass obj;

    AnotherClass(MyClass o) {
        obj = o;
    }
}
```

```

class MyClass {
    int data = 50;
}

public class Test {
    public static void main(String[] args) {
        MyClass a = new MyClass();
        AnotherClass b = new AnotherClass(a);
        System.out.println(b.obj.data);
    }
}

```

Output:

50

Constructor overloading

Constructor Overloading is an object-oriented programming concept in which a class contains two or more constructors with the same name but different parameter lists. The constructors differ in the number, type, or order of parameters, allowing the creation of objects in multiple ways.

The appropriate constructor is automatically selected by the compiler based on the arguments passed during object creation. Constructor overloading improves flexibility, clarity, and reusability of code by enabling different initialization scenarios for the same class.

Example:

```

class MyClass {
    int data;
    String name;
    double value;

    MyClass() {
        data = 0;
        name = "default";
        value = 0.0;
    }

    MyClass(int d) {
        data = d;
        name = "default";
        value = 0.0;
    }
}

```

```
MyClass(int d, String n) {  
    data = d;  
    name = n;  
    value = 0.0;  
}  
  
MyClass(int d, String n, double v) {  
    data = d;  
    name = n;  
    value = v;  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyClass obj = new MyClass(5, "Test", 9.5);  
        System.out.println(obj.data + " " + obj.name + " " + obj.value);  
    }  
}
```

Output:

5 Test 9.5